GIRLY ML · PYTHON CHEAT SHEET

Level 1
Foundations & EDA

The building blocks — functions, data structures, and your first pass at understanding a new dataset.

● 01 / 03 — start here

Functions & Loops

What it is: reusable blocks of logic (functions) and repeated execution (loops) — the core mechanics of any script.

Used for: avoiding repeated code, processing every row/item in a collection.

```
def clean_price(x):
    return float(x.replace('$', ''))

for col in df.columns:
    print(col, df[col].dtype)
```

EXAMPLE — ENUMERATE + ZIP

```
for i, name in enumerate(names):
    print(i, name)

for name, score in zip(names, scores):
    print(f"{name}: {score}")
```

Core Data Structures

What it is: lists (ordered, mutable), dicts (key-value pairs), tuples (ordered, immutable), and sets (unique values) — how Python holds data before it's a DataFrame.

Used for: quick lookups, deduplication, building records before loading into pandas.

```
scores = [92, 85, 78]
student = {'name': 'Mia', 'gpa': 3.8}
unique_ids = set(df['id'])
```

EXAMPLE — COMPREHENSIONS

```
squared = [x**2 for x in scores]
passing = {k: v for k, v in grades.items() if v >= 60}
```

pandas Basics

What it is: the standard way to load and get a first read on tabular data — shape, types, and summary stats before you touch anything.

Used for: the very first step of any analysis, right after loading a file.

```
df = pd.read_csv('sales.csv')
df.info()
df.describe()
df.shape
```

EXAMPLE — QUICK COLUMN-LEVEL CHECKS

```
df.dtypes
df.columns.tolist()
df.head(10)
```

First-Pass EDA

What it is: a structured scan for the three things that break analyses — missing values, wrong data types, and unexpected category counts.

Used for: deciding what needs cleaning before any real analysis starts.

```
df.isnull().sum()
df['category'].value_counts()
df.dtypes
```

EXAMPLE — % MISSING PER COLUMN

```
(df.isnull().sum() / len(df) * 100).round(1)
```

Quick Viz

What it is: fast, throwaway plots to see shape and spread — not polished, just diagnostic.

Used for: spotting skew, outliers, and category imbalance before deeper analysis.

```
import matplotlib.pyplot as plt
df['revenue'].hist(bins=30)
df.boxplot(column='revenue', by='region')
```

EXAMPLE — QUICK SCATTER

```
plt.scatter(df['ad_spend'], df['revenue'])
plt.xlabel('Ad Spend'); plt.ylabel('Revenue')
```

Correlation Basics

What it is: a quick descriptive check of how two numeric variables move together, from -1 (perfectly opposite) to 1 (perfectly together). Doesn't prove causation.

Used for: a first pass at "which variables might be related" before formal testing.

```
df.corr(numeric_only=True)
```

EXAMPLE — CORRELATION HEATMAP

```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='RdPu')
```

Level 2
Real-World Wrangling

Data doesn't arrive as a clean CSV — pulling it from APIs, parsing JSON, and shaping it into something usable.

● 02 / 03 — leveling up

Working with JSON

What it is: a nested key-value format common in APIs and logs — not naturally tabular, so it needs flattening before it's analysis-ready.

Used for: API responses, config/log data, semi-structured records.

```
import json
with open('data.json') as f:
    data = json.load(f)
df = pd.json_normalize(data)
```

EXAMPLE — FLATTENING NESTED FIELDS

```
df = pd.json_normalize(
    data,
    record_path='orders',
    meta=['customer_id', 'signup_date']
)
```

Pulling Data from APIs

What it is: requesting data over HTTP instead of reading a static file — the source of most "live" or third-party datasets.

Used for: pulling fresh data on a schedule, working with services that don't offer file exports.

```
import requests
r = requests.get(
    'https://api.example.com/sales',
    headers={'Authorization': f'Bearer {token}'}
)
data = r.json()
```

EXAMPLE — HANDLING PAGINATION

```
results = []
url = 'https://api.example.com/sales?page=1'
while url:
    r = requests.get(url).json()
    results += r['data']
    url = r.get('next_page')
```

Multiple File Formats

What it is: pandas readers/writers for the formats you'll actually encounter beyond CSV — each with its own quirks (sheet names, engines, compression).

Used for: pulling from Excel workbooks, databases, and efficient columnar storage.

```
pd.read_excel('report.xlsx', sheet_name='Q3')
pd.read_sql('SELECT * FROM sales', conn)
pd.read_parquet('events.parquet')
```

EXAMPLE — WRITING OUT CLEANED DATA

```
df.to_parquet('clean_sales.parquet', index=False)
df.to_excel('report.xlsx', sheet_name='clean', index=False)
```

GroupBy + Agg + Merge

What it is: combining and summarizing data across tables and categories — the workhorse operations of real analysis.

Used for: per-group KPIs, joining customer/order/product tables together.

```
df.groupby('region').agg(
    total_rev=('revenue', 'sum'),
    orders=('order_id', 'count')
)
pd.merge(orders, customers, on='customer_id', how='left')
```

EXAMPLE — CONCATENATING MONTHLY FILES

```
all_months = pd.concat(
    [pd.read_csv(f) for f in file_list],
    ignore_index=True
)
```

Regex & String Cleaning

What it is: pattern matching for messy text columns — inconsistent formatting, embedded info, typos.

Used for: extracting structured info from free text, standardizing categorical labels.

```
df['domain'] = df['email'].str.extract(r'@(.+)')
df['clean_phone'] = df['phone'].str.replace(r'\D', '', regex=True)
```

EXAMPLE — FILTERING WITH STR.CONTAINS

```
df[df['notes'].str.contains('refund', case=False, na=False)]
```

Intro Time Series

What it is: treating dates as a real index rather than plain text, which unlocks date-based filtering and resampling.

Used for: converting daily data to weekly/monthly rollups, plotting trends over time.

```
df['date'] = pd.to_datetime(df['date'])
df = df.set_index('date')
monthly = df['revenue'].resample('M').sum()
```

EXAMPLE — EXTRACTING DATE PARTS

```
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.day_name()
```

GIRLY ML · PYTHON CHEAT SHEET

Level 3
Advanced Analytics

Where analysis gets rigorous — proper time series decomposition, statistical testing, and code built to be reused.

● 03 / 03 — advanced

Web Scraping

What it is: pulling structured data directly out of HTML when no API or file export exists.

Used for: pricing data, public directories, research datasets with no official export.

```
from bs4 import BeautifulSoup
r = requests.get(url)
soup = BeautifulSoup(r.text, 'html.parser')
prices = [p.text for p in soup.select('.price')]
```

EXAMPLE — BUILDING ROWS INTO A DATAFRAME

```
rows = []
for item in soup.select('.product'):
    rows.append({
        'name': item.select_one('.name').text,
        'price': item.select_one('.price').text
    })
df = pd.DataFrame(rows)
```

Time Series Analysis

What it is: going beyond resampling into trend, seasonality, and momentum — understanding not just "what happened" but "what pattern is repeating."

Used for: forecasting inputs, detecting seasonality, momentum-based features.

```
df['rolling_avg'] = df['revenue'].rolling(7).mean()
df['mom_change'] = df['revenue'].diff()
```

EXAMPLE — SEASONAL DECOMPOSITION + AUTOCORRELATION

```
from statsmodels.tsa.seasonal import seasonal_decompose
result = seasonal_decompose(df['revenue'], period=12)
result.plot()

from statsmodels.graphics.tsaplots import plot_acf
plot_acf(df['revenue'])
```

Statistical Testing

What it is: formal tests that quantify whether a pattern is real or just noise, using a p-value against a significance threshold (commonly 0.05).

Used for: A/B test results, comparing group means, testing relationships between categorical variables.

```
from scipy import stats
t_stat, p_val = stats.ttest_ind(group_a, group_b)
chi2, p_val, dof, expected = stats.chi2_contingency(table)
```

EXAMPLE — ONE-WAY ANOVA + CORRELATION P-VALUE

```
f_stat, p_val = stats.f_oneway(group_a, group_b, group_c)
r, p_val = stats.pearsonr(df['ad_spend'], df['revenue'])
```

Feature Engineering for ML

What it is: transforming raw columns into model-ready inputs — scaling, encoding, and splitting data properly.

Used for: prepping a cleaned dataset to hand off to a model without leaking test data into training.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train_scaled = StandardScaler().fit_transform(X_train)
```

EXAMPLE — ONE-HOT ENCODING CATEGORICALS

```
df_encoded = pd.get_dummies(df, columns=['region', 'plan_type'])
```

Reusable Pipelines

What it is: packaging repeated logic into functions and classes instead of copy-pasting notebook cells — code that survives being run twice.

Used for: repeatable EDA reports, standardized cleaning steps across projects.

```
class DataCleaner:
    def __init__(self, df):
        self.df = df

    def drop_nulls(self, thresh=0.5):
        return self.df.dropna(thresh=len(self.df.columns)*thresh)
```

EXAMPLE — TRY/EXCEPT FOR ROBUST PIPELINES

```
try:
    df['price'] = df['price'].astype(float)
except ValueError as e:
    print(f"Bad price value: {e}")
```

Performance

What it is: making code fast enough for large datasets — pandas' vectorized operations run in optimized C, while Python-level loops are slow row-by-row.

Used for: scripts that time out or crawl on real-sized data (100k+ rows).

```
# slow — Python loop
df['total'] = [r.qty * r.price for r in df.itertuples()]

# fast — vectorized
df['total'] = df['qty'] * df['price']
```

EXAMPLE — MEMORY OPTIMIZATION WITH DTYPES

```
df['region'] = df['region'].astype('category')
df['qty'] = pd.to_numeric(df['qty'], downcast='integer')
# .query() is often faster than chained boolean masks
df.query('revenue > 1000 and region == "East"')
```